

AMSS 2026 — Lecture 7: Behavioral II — State + Activity

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from one interaction to a whole lifecycle
2. Two views: state machines + activity diagrams
3. Live opener: drive AI to a Bike state machine
4. Reading critically: orphan states, dead ends, missed guards
5. The behaviour read-order
6. Bridge to Week 8 + Lab 4

Recap: Architect and Critic

- ▶ **Architect / director:** drive AI through the software development lifecycle (SDLC).
- ▶ **Critic / reviewer:** read AI's output and name what's wrong or missing.

In Week 6 you critiqued an AI *interaction* (a sequence). Today: the same loop, on a *lifecycle* and a *workflow*.

From One Interaction to a Whole Lifecycle

- ▶ **Week 6 pinned one interaction** — the messages for “rent a bike”, one path through one use case.
- ▶ **Today: the whole behaviour** — a Bike lives through many **states**; a rental **workflow** branches and runs in parallel.

Today's question: **can the object actually reach — and leave — every state the model claims?**

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- ▶ **Critique** — read & critique AI-generated artifacts on the spot.
- ▶ **Rationale** — articulate why you directed AI a certain way.
- ▶ **Traceability** — defend the trace across your project.

Today is Critique on lifecycles: read a state machine and name orphan states, unreachable transitions, missed guards. **Traceability touch:** state and activity are behavioural nodes in the trace, alongside the Week 6 sequence.

Two Views Today

Both answer “how does it behave?” at different scopes:

- ▶ **State machine** — one object’s **lifecycle**: the states it passes through and what moves it between them.
- ▶ **Activity diagram** — a **workflow** across steps (and actors): branches, parallelism, start to finish.

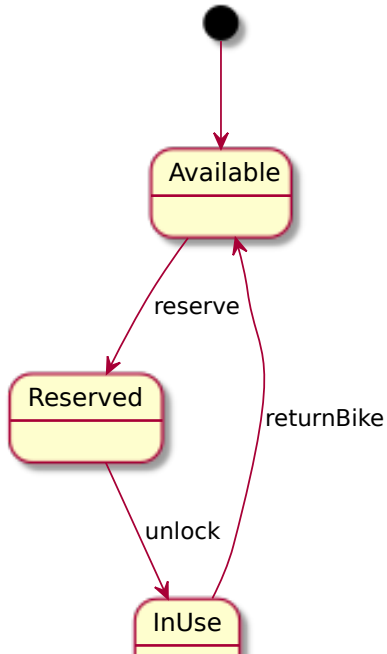
The named payload is **state-machine defects**; both views are load-bearing for Lab 4.

Demo: Drive AI to a Bike State Machine

Live: ask AI for the lifecycle of a Bike. Watch for a state it can never reach, a state it can never leave, and a branch with no guard.

Prompt to AI: *“Generate a UML state machine diagram (as PlantUML) for a bike in the city bike-sharing app: it can be available, reserved, in use, and under maintenance.”*

State Machine: States + Transitions



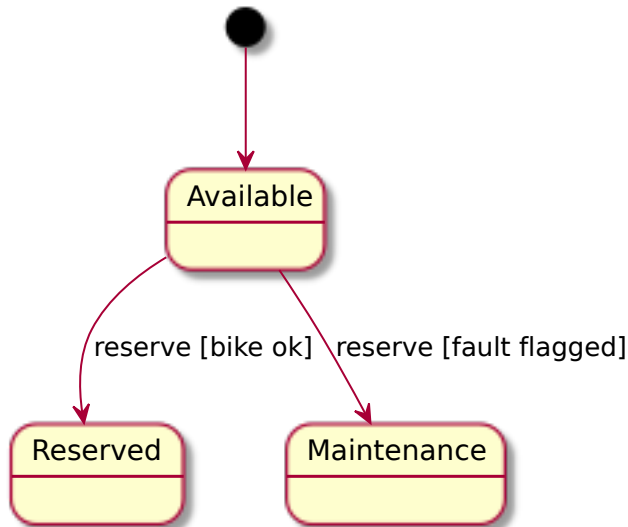
Reading a State Machine

Read every state, and ask:

1. **Reachable?** Is there a path from the initial state to this one?
2. **Escapable?** Does this state have a way out — or is it a final state?
3. **Triggered?** Does every transition name the event that fires it?

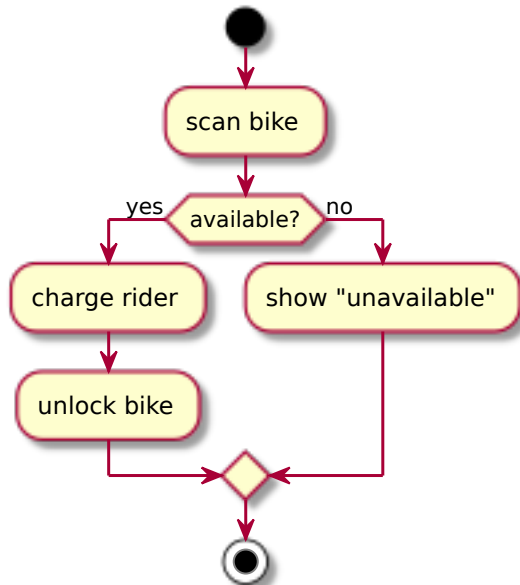
A state you can't reach, or can't leave, is a defect.

Guards + Events



A **guard** [condition] chooses between transitions on the **same event**. Without guards, two transitions on reserve are ambiguous

Activity Diagram: Actions + Flow



Bounded boxes are **actions**; the diamond is a **decision**; flow runs

Decisions, Forks, Joins

- ▶ **Decision / merge** — a branch (diamond out) must rejoin (merge in).
- ▶ **Fork / join** — parallel flows split at a fork bar and must synchronise at a join bar.
- ▶ **One initial**; one or more **final** nodes.

Every split needs its matching rejoin.

Reading an Activity

Read the flow, and ask:

1. Does every **decision** eventually **merge**?
2. Does every **fork** reach a **join**?
3. Is there a path from start to a **final** node?

A dangling branch or an unsynchronised fork is a defect.

AI Draws Plausible Lifecycles — and Strands States

A state machine can list all the right states and still be broken: a state nothing reaches, a state nothing leaves, a branch nothing decides. The critic question:

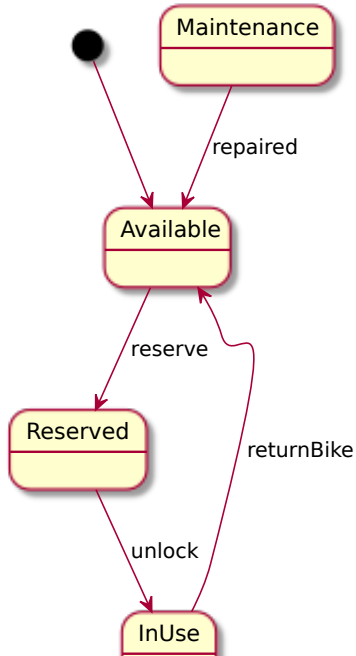
“Can the object actually reach — and leave — every state, and is every branch decided?”

Your Turn: Spot the Defect

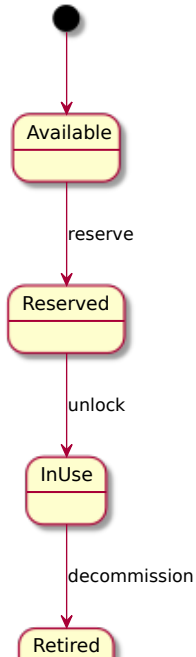
Look at the next state machine before we critique it.

20 seconds with your neighbour: what's wrong with this lifecycle before we critique it?

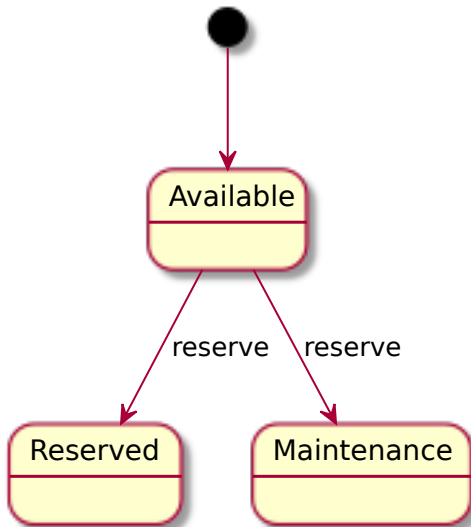
Defect #1: Orphan / Unreachable State



Defect #2: Dead-End State

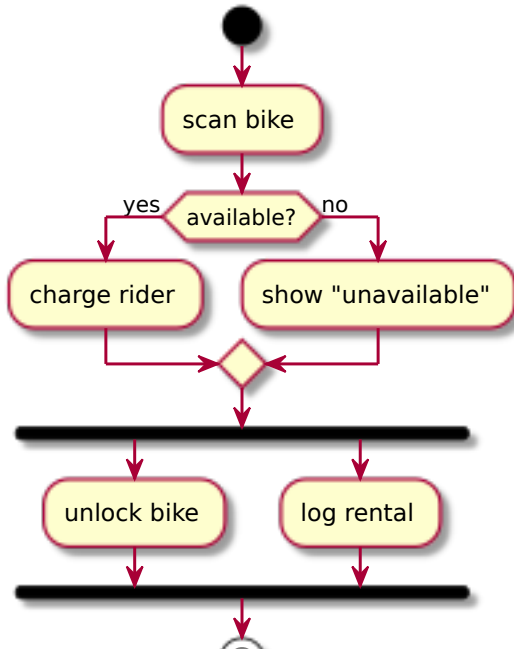


Defect #3: Missed Guard



Two reserve transitions, no guards — which fires? **Critique:**
“Both on the same event with no condition. What decides? Add the

Defect #4: Activity — Missing Merge / Fork Without Join

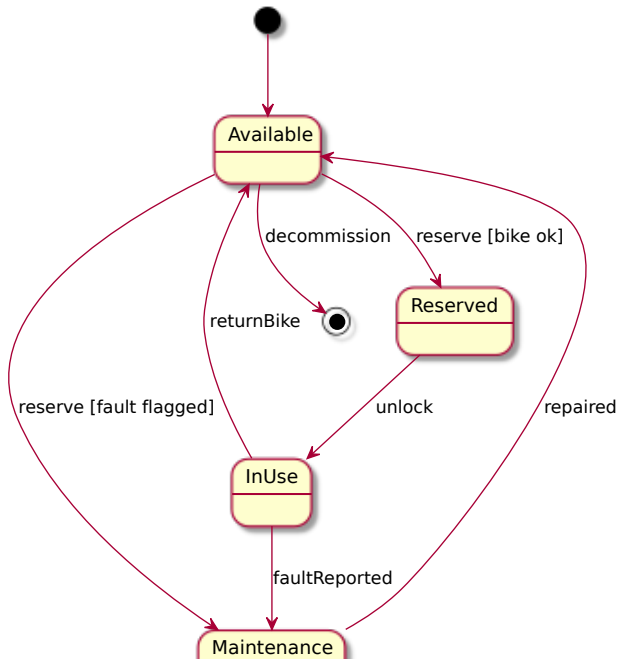


Defect #5: No Initial / No Final

A state machine with no initial pseudo-state (where does it begin?), or an activity with no final node (when is it done?).

Critique: *“Where does this start? Where does it end? A behaviour model needs both.”*

The Critic's Corrected State Machine



How to Read an AI Behaviour Model

A fixed order, every time:

1. Is every **state / node reachable** from the start?
2. Is every state **escapable** (or a final)?
3. Are there **guards** wherever one event branches?
4. Does every **decision merge** and every **fork join**?
5. Is there a path to a **final**?

The Critique Loop on Lifecycles

read -> name the orphan/unreachable/missed-guard -> re-prompt with the missing transitions + guards -> re-read.

Same architect-and-critic loop as Weeks 2-6 — now on a lifecycle and a workflow.

The Human Decides the Real Lifecycle

AI can list states and draw transitions forever. Whether the object can *actually* move through them — reach every state, leave every state, branch correctly — is the architect-and-critic call.

It's what the oral defense checks, and AI can't make it for you.

Looking Ahead: Lab 4 (Next Week)

Lab 4 (Week 8) is a **team defect hunt on flawed behavioural artifacts** — sequence (Week 6) + state + activity (Week 7), same format as Lab 3.

Today's behaviour read-orders are your defect-hunt rubric.

Next Week: Patterns I (Week 8)

Behaviour is done — Week 6 (interactions) and Week 7 (lifecycles + workflows) cover *how the system behaves*.

Week 8: design patterns — the reusable solutions, when to apply them, and how AI overuses or mislabels them. (Plus Lab 4, the behavioural defect hunt.)

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- ▶ **Critique** — read an AI state machine and named its orphan states, dead ends, and missed guards.
- ▶ **Traceability** — state and activity join the sequence as behavioural nodes in the full trace: requirement -> use case -> class -> sequence / state / activity -> test.

Rationale (why you directed AI a certain way) lands in your project narrative.

That's It For Today

- ▶ Next lecture (Week 8): patterns I — selection.
- ▶ Next week's lab (Lab 4): team defect hunt on flawed behavioural artifacts.

Questions?